

Agentic Memory through the Lens of Incremental View Updates

From hard-coded memory design to a declarative, optimized memory substrate

May 21, 2026

1 Introduction

1.1 Business and Technical Problems

Agent memory today is mostly one-off and application-specific: each product owns its own memory logic, which leads to duplicated engineering, divergent user experiences across products, and slow, fragmented rollout of new memory features. Even when a product uses a third-party service, such as TencentDB AI-Memory, the underlying memory schema and logic are often fixed. Product teams therefore cannot flexibly define their own memory structures and memory policies.

Companies lack a shared and configurable memory substrate. This slows the rollout of new memory capabilities and limits cross-product reuse.

The technical problem is that most existing agent-memory implementations follow a fixed-schema plus hand-written-pipeline pattern. Every new scenario, agent, task, or product requires redesigning schemas and update logic. This is costly and brittle. Memory behavior is hidden in code, which prevents systematic optimization of token cost, latency, and freshness tradeoffs.

Emerging multi-agent and multi-user settings make the problem sharper. Memory needs to become a shared platform with privacy, permission, and scalability guarantees, rather than isolated state inside a single agent. Current systems also tend to use eager per-message updates, which makes it difficult to support policy choices such as always-fresh memory for VIP users and lower-cost lazy updates for long-tail users in one unified framework.

1.2 Research Gap

Both research and industry are advancing quickly, but most systems still follow the same fixed-schema plus hand-written-pipeline pattern.

TencentDB Agent Memory shows that layered memory can reduce token usage and improve task completion, but its memory hierarchy and update flow are still designed as a predefined system rather than as a general declarative substrate. OpenViking improves persistent context by organizing memory as a hierarchical file system with tiered loading, yet it still relies on a built-in structure for context management rather than exposing memory as a queryable, optimizable view. Claude Auto Memory can automatically capture useful information across sessions, but it remains constrained by a fixed memory structure rather than a user-programmable view layer. Zep studies structured long-term conversational memory, but it also assumes a built-in memory organization rather than a declarative memory view. Mem0 explores selective memory formation for personalized agents, but its update behavior remains pipeline-centric rather than automatically derived from a user-defined specification.

The key gap is not whether memory exists. The key gap is whether memory can become a programmable data substrate: declarative in memory design, efficient in memory maintenance, and tunable in runtime policy.

1.3 Proposal and Objectives

We propose a reusable memory substrate where developers define memory structures and policies through a declarative interface instead of hand-coding low-level update logic. The core abstraction treats messages as a base table and memory as a view over that table. Memory design becomes an explicit, query-like specification rather than hidden application logic.

The project has three objectives:

1. Provide a concise API for declaratively defining agent memory as views, expressed in Pandas-style queries Q extended with LLM-based semantic operators such as semantic filtering, semantic mapping, semantic grouping, semantic joining, and semantic top-k.
2. Build an incremental view-maintenance optimizer that automatically derives the corresponding maintenance queries Q' from Q , eliminating the need for developers to hand-craft and maintain update logic.
3. Build a runtime execution engine that schedules and executes Q' when new messages arrive, choosing eager, lazy, or batched plans that reduce token usage while exposing tunable tradeoffs between token cost, latency, and view freshness.

1.4 Memory as DataFrame Views

Our starting point is that agent memory should be treated as a database problem, where messages are tuples in a base table. Different memory behaviors, such as short-term context buffers, long-term user profiles, topic clusters, catalogs, entities, facts, and relations, can be expressed as different view definitions over the same message stream.

For example, Claude-style memory organizes durable memory around topics. It extracts candidate topics from conversations, merges related candidates into stable memories, and creates a catalog entry for each topic so the system can retrieve them efficiently later. In the current DataFrame-style API, the topic memory and catalog can be written as two views:

```
topic_candidates = (
    log
    .sem_flat_map(
        input_cols=["message"],
        output_cols={
            "topic_name": "Candidate durable memory topic name.",
            "topic_content": "Candidate durable memory content.",
        },
        instruction=(
            "Extract zero or more durable memory topic candidates from "
            "{message}, filling {topic_name} and {topic_content}."
        ),
    )
    .select(["topic_name", "topic_content"])
```

```

)

topics = topic_candidates.sem_groupby(
    key=["topic_name"],
    instruction="Find candidates related to the same durable memory topic.",
    agg=sem_agg(
        input_cols=["topic_name", "topic_content"],
        output_cols={
            "topic_name": "Canonical durable memory topic name.",
            "topic_content": "Merged durable memory content.",
        },
        instruction="Choose a canonical topic name and merge topic content.",
    ),
)

catalog = (
    topics
    .sem_map(
        input_cols=["topic_name", "topic_content"],
        output_cols={
            "catalog_title": "Title shown in the memory catalog.",
            "path": "Relative path to the topic memory file.",
            "hook": "One-line relevance hook for future retrieval.",
        },
        instruction="Produce one catalog row per topic.",
    )
    .select(["catalog_title", "path", "hook"])
)

```

The important property is that users write the full logical view definition Q . They do not write the low-level maintenance pipeline.

1.5 From Q to Q'

In traditional databases, a view is defined as

$$V = Q(D),$$

where Q is a query over base data D . When a new message ΔD arrives, the naive strategy is to recompute

$$V' = Q(D \cup \Delta D).$$

This is wasteful because it reprocesses the entire history D .

Existing memory systems such as Claude, Zep, and Mem0 avoid this cost through manually designed update logic. Their developers handcraft a new-message handling pipeline that updates only the necessary data in memory. This ad-hoc approach can work for one product, but it does not scale to arbitrary personalized memory designs.

The central research question is:

Given a memory view $V = Q(D)$, how do we derive a practical, systematic, and theoretically grounded maintenance query Q' such that, when new data ΔD arrives, the system updates the view using Q' instead of recomputing $Q(D \cup \Delta D)$?

The maintenance query Q' operates on small new data and maintained view state, so it can be much faster in user latency and much cheaper in token cost. Without semantic operators, this is the classical incremental view maintenance problem. Our setting is simpler than modern distributed dataflow systems because user memory is low-throughput, local, and privacy-sensitive. It is also harder in a new way: semantic operators powered by LLMs are non-deterministic, cost-sensitive, and prompt-dependent.

At the operator level, the project derives differential forms for semantic operators such as semantic group-by, semantic join, and semantic top-k. These differential forms become the basis for an optimizer that can choose cheaper execution plans while preserving memory quality.

1.6 Runtime Tradeoff and Milestones

Maintaining agent memory creates a three-way tradeoff among LLM token cost, user-facing latency, and view freshness. Eager updates maximize freshness and therefore query-time accuracy, but they incur token cost on every incoming message. Lazy and batched updates can be much cheaper because they amortize prompting overhead across multiple messages and can exploit discounted batch inference. The cost of this efficiency is temporary staleness: queries issued before the next update may read an older view.

The runtime execution engine should make this tradeoff explicit and adjustable at runtime, so that different memory policies, such as low-cost but slightly stale memory or always-fresh memory for VIP users, can be tuned per user or per product rather than baked into code.

The planned milestones are:

1. Release the API for declaratively defining agent memory, and validate it by reproducing Claude-style and Zep-style memory.
2. Release the incremental view-maintenance optimizer that automatically derives maintenance queries from view definitions.
3. Release the execution engine that schedules eager, lazy, or batched execution of Q' when new messages arrive.

The intended outcome is a cost-efficient, reusable memory substrate that supports existing and new agent products, together with empirical evidence from benchmarks or A/B studies that declarative, view-based memory can match or surpass hand-built pipelines in development effort and performance metrics.

2 Relational and Semantic Delta Rules

The rules below separate the full view definition Q , the derived maintenance query Q' , and the physical runtime plan. Relations, views, and intermediate results are dataframe-like tables. Unless stated otherwise, ΔD denotes new input rows.

2.1 Semantic Filtering

For a semantic filter,

$$Q(D) = V = D.\text{sem_filter}(\text{instruction}).$$

The maintenance query only filters the changed rows:

$$Q'(V, \Delta D) = V \cup \Delta D.\text{sem_filter}(\text{instruction}).$$

The semantic uncertainty is in the filter decision itself. Useful physical plans include prompt caching, batching, predicate pushdown, and cheaper models for low-risk rows.

2.2 Projection, Map, and FlatMap

Classical relational algebra has projection. DataFrame APIs expose projection through column selection, such as `select` or bracket selection. In the current operator API, `sem_map` and `sem_flat_map` are add-column operators. They preserve source columns and add or replace requested output columns. A new logical table is produced by applying ordinary projection afterward.

For semantic map:

$$Q(D) = D.\text{sem_map}(\text{input_cols}, \text{output_cols}, \text{instruction}).$$

The row-local maintenance rule is:

$$Q'(V, \Delta D) = V \cup \Delta D.\text{sem_map}(\text{input_cols}, \text{output_cols}, \text{instruction}).$$

If the full view definition projects columns, the differential branch must apply the same projection before union:

```
V = (
  D
  .sem_map(output_cols=[...], instruction="...")
  .select(V_columns)
)

V_prime = V.union(
  delta_D
  .sem_map(output_cols=[...], instruction="...")
  .select(V_columns)
)
```

For semantic flat-map, one input row may emit zero, one, or many output rows:

$$Q'(V, \Delta D) = V \cup \Delta D.\text{sem_flat_map}(\text{input_cols}, \text{output_cols}, \text{instruction}).$$

Again, any projection in the full view definition must be mirrored on the delta branch.

2.3 Semantic GroupBy with Aggregation

The current public API treats semantic group-by as semantic grouping plus an aggregate expression:

```
V = D.sem_groupby(  
    key=[...],  
    instruction=group_instruction,  
    agg=agg,  
)
```

Here `key` names the columns used as the grouping basis, the `instruction` defines semantic membership, and `agg` produces the output row for each group. The aggregate can be ordinary, such as `count()` or `max("timestamp")`, or semantic, such as `sem_agg(...)`.

A coarse full-next-view maintenance rule is:

```
delta_groups = delta_D.sem_groupby(  
    key=[...],  
    instruction=group_instruction,  
    agg=agg,  
)  
  
V_prime = (  
    delta_groups  
    .sem_join(V, join_instruction_prime, how="outer")  
    .sem_map(  
        output_cols=V.columns,  
        instruction="""  
        Produce one next-view row:  
        merge matched delta group and existing view row;  
        keep unmatched existing view row;  
        add unmatched delta group row.  
        """,  
    )  
    .select(V.columns)  
)
```

This is a coarse rule, not the only possible physical plan. The final `sem_map` is a column-level semantic merge over joined rows, not a `sem_agg`. A more optimized plan may split exact `subtract` and `union` effects, but that is a lower-level maintenance plan rather than the primary author-facing view definition.

2.4 Semantic Aggregation

Standalone semantic aggregation turns many rows into one row:

$$Q(D) = D.\text{sem_agg}(\text{input_cols}, \text{output_cols}, \text{instruction}).$$

The `input_cols` are the read set and the `output_cols` are the write set. They are not positional rename lists and do not need to be one-to-one. For example, many message and timestamp rows can produce one summary and date-range row.

A compressed-state maintenance plan is:

$$Q'(V, \Delta D) \approx (V \cup \Delta D). \text{sem_agg}(\text{input_cols}, \text{output_cols}, \text{instruction}).$$

This is exact only when V is a sufficient aggregate state for the instruction. Otherwise it is an approximation because the old aggregate row may have lost information from the original input rows.

2.5 Semantic Join

For semantic join,

$$Q(L, R) = L. \text{sem_join}(R, \text{instruction}, \text{how}).$$

If both sides can change, the maintenance query follows the usual relational delta shape:

$$\begin{aligned} Q' &= \Delta L. \text{sem_join}(R, \text{instruction}, \text{how}) \\ &\cup L. \text{sem_join}(\Delta R, \text{instruction}, \text{how}) \\ &\cup \Delta L. \text{sem_join}(\Delta R, \text{instruction}, \text{how}). \end{aligned}$$

Each branch can be physically lowered to candidate retrieval, approximate nearest-neighbor search, embedding search, BM25, graph traversal, reranking, or hybrid retrieval before applying the semantic join predicate. The public API can use either `sem_join(..., how=...)` or convenience aliases such as `sem_left_join`, but the canonical documentation form is `sem_join(..., how=...)`.

2.6 Semantic Top-k

For semantic top-k,

$$Q(D, x) = V = D. \text{sem_topk}(x, k).$$

If D changes but x is fixed, the insert-only maintenance rule can compare scored changed rows against the existing top-k boundary:

$$Q'(V, \Delta D, x) \approx (V \cup \text{sem_score}(\Delta D, x)). \text{top}_k(k).$$

The scoring function may be implemented with vector search, keyword search, hybrid search, graph traversal, reranking, or LLM scoring. Those choices are `sem_topk` optimization strategies, not separate author-facing operators.

If x changes, for example because x is another upstream view, then exact maintenance usually requires re-retrieval or rescoring over D :

$$Q'_x = D. \text{sem_topk}(x + \Delta x, k).$$

3 Examples

3.1 Claude-Style Topic Memory

The full view definition is:

```
topic_candidates = (
    log
    .sem_flat_map(
```

```

        input_cols=["message"],
        output_cols=["topic_name", "topic_content"],
        instruction="Extract durable topic candidates from {message}.",
    )
    .select(["topic_name", "topic_content"])
)

topics = topic_candidates.sem_groupby(
    key=["topic_name"],
    instruction="Find candidates related to the same durable memory topic.",
    agg=sem_agg(
        input_cols=["topic_name", "topic_content"],
        output_cols=["topic_name", "topic_content"],
        instruction="Choose a canonical topic name and merge topic content.",
    ),
)

catalog = (
    topics
    .sem_map(
        input_cols=["topic_name", "topic_content"],
        output_cols=["catalog_title", "path", "hook"],
        instruction="Produce one catalog row per topic.",
    )
    .select(["catalog_title", "path", "hook"])
)

```

The coarse maintenance query for topics is:

```

delta_topic_candidates = (
    delta_log
    .sem_flat_map(
        input_cols=["message"],
        output_cols=["topic_name", "topic_content"],
        instruction="Extract durable topic candidates from {message}.",
    )
    .select(["topic_name", "topic_content"])
)

delta_groups = delta_topic_candidates.sem_groupby(
    key=["topic_name"],
    instruction="Find candidates related to the same durable memory topic.",
    agg=sem_agg(
        input_cols=["topic_name", "topic_content"],
        output_cols=["topic_name", "topic_content"],
        instruction="Choose a canonical topic name and merge topic content.",
    ),
)

```

```

topics_prime = (
    delta_groups
    .sem_join(
        topics,
        "Find related existing topics, including corrections, "
        "contradictions, supersession, and forget targets.",
        how="outer",
    )
    .sem_map(
        output_cols=["topic_name", "topic_content"],
        instruction="Produce the next topic row for each joined row.",
    )
    .select(["topic_name", "topic_content"])
)

```

The catalog can be recomputed from `topics_prime` at the logical level, or the runtime can fuse catalog production with topic maintenance and only run semantic catalog mapping for changed topic rows.

3.2 Zep-Style Entity and Relation Memory

Zep-style memory can be described as views over the same log:

```

episodes = log.sem_map(
    input_cols=["message"],
    output_cols=["episode_id", "content", "timestamp"],
    instruction="Wrap each message as a stored episode row.",
).select(["episode_id", "content", "timestamp"])

entity_candidates = (
    episodes
    .sem_flat_map(
        input_cols=["content"],
        output_cols=["entity_name", "entity_type", "evidence"],
        instruction="Extract entity candidates from each episode.",
    )
    .select(["entity_name", "entity_type", "evidence"])
)

entities = entity_candidates.sem_groupby(
    key=["entity_name"],
    instruction="Find candidates that refer to the same real-world entity.",
    agg=sem_agg(
        input_cols=["entity_name", "entity_type", "evidence"],
        output_cols=["entity_name", "entity_type", "summary"],
        instruction="Choose a canonical entity and summarize evidence.",
    ),
)

```

Relations and facts follow the same pattern: semantic flat-map extracts candidates, semantic group-by resolves duplicates, and semantic aggregation produces canonical rows.

3.3 Mem0-Style Fact Memory

Mem0-style memory can be expressed as a fact view:

```
fact_candidates = (
    log
    .sem_flat_map(
        input_cols=["message"],
        output_cols=["fact", "subject", "evidence"],
        instruction="Extract durable memory facts from each message.",
    )
    .select(["fact", "subject", "evidence"])
)

facts = fact_candidates.sem_groupby(
    key=["fact", "subject"],
    instruction="Find candidates that represent the same durable memory fact.",
    agg=sem_agg(
        input_cols=["fact", "subject", "evidence"],
        output_cols=["fact", "subject", "summary"],
        instruction="Merge fact evidence into one canonical fact row.",
    ),
)
```

The runtime can lower this logical query into add, update, and delete effects, but the author-facing policy remains the full view definition.

4 Maintenance Strategy

Q' defines how a view can be maintained. A maintenance strategy decides when and under what budget to run it.

4.1 Eager or Continuous Maintenance

Eager maintenance runs Q' as soon as new messages arrive. It maximizes freshness and usually improves query-time accuracy, but it pays update cost on every update.

4.2 Deferred or Lazy Maintenance

Lazy maintenance delays Q' until a later time, a larger batch, or a query that needs the view. This can reduce token cost by amortizing prompt overhead, but it introduces temporary staleness.

4.3 Batching and Query-Triggered Repair

Batching processes ΔD_{batch} rather than one row at a time. This can reduce per-row overhead and take advantage of batch inference, but batch maintenance must handle interactions among delta rows. Query-triggered repair runs maintenance only when the stale view would affect a query result.

4.4 Full Rebuild and Delta Repair

Incremental maintenance can accumulate semantic drift because LLM outputs are approximate. The system should support full rebuilds or partial repair when estimated drift exceeds a threshold, when prompts change, or when the store representation changes.

5 Partial Materialization and Cuboid Lattices

Incremental view maintenance optimizes how to update chosen views. A separate question is which views should be maintained at all.

Classical OLAP systems study this through data cubes and cuboid lattices. A base fact table can be aggregated along multiple dimensions. Each cuboid is a materialized aggregate at a different granularity. Maintaining all cuboids is fast for queries but expensive for update and storage. Maintaining none is cheap for updates but slow for queries. Partial materialization chooses a subset of cuboids under cost and latency constraints.

Semantic memory has an analogous problem. Possible memory views include topic files, catalogs, entity tables, fact tables, relation tables, summaries, communities, and retrieval indexes. Maintaining more views can reduce query latency and improve retrieval. It also costs more to update and may introduce information loss because semantic views summarize, merge, or canonicalize the underlying log.

The semantic-memory version of materialization selection must therefore optimize both cost and accuracy:

$$\min_m C_{\text{update}}(m) + C_{\text{query}}(m) + C_{\text{store}}(m) + \lambda L_{\text{quality}}(m) + \mu L_{\text{staleness}}(m).$$

This remains a research direction. The useful conclusion is that not every derived semantic view should be maintained, and choosing a maintained view is an accuracy decision as well as a cost decision.

6 Research Agenda

1. **Semantic Q' derivation.** Given a logical semantic view query Q , generate candidate maintenance queries $Q'_1, \dots, Q'_k$. Plans may differ in grouping strategy, join candidate retrieval, compressed-state aggregation, batching, model choice, and repair strategy.
2. **Operator-specific rules.** Identify which operators behave like row-local operators and which require affected-state retrieval. `sem_filter`, `sem_map`, and `sem_flat_map` are local. Semantic group-by with aggregation and `sem_join` require old view state or candidate retrieval.
3. **Cost and quality models.** Estimate update cost, query cost, false merge rate, missed merge rate, contradiction risk, and summarization loss for each maintenance plan.
4. **Lazy update under quality constraints.** Design schedulers that defer expensive maintenance without letting user-perceived query quality fall below a threshold.
5. **Materialization selection.** Adapt cuboid and materialized-view selection to semantic views, where materialization changes accuracy as well as cost.
6. **Top-k planning.** Decide how `sem_topk` should lower into scoring, candidate generation, order-by, limit, and boundary maintenance when a top-k result is materialized.

7 Pointers

- Current proposal source: `docs/draft.txt`.
- Current operator API: `docs/design/operator-api.md`.
- Internal IVM reference: `docs/reference/ivm.md`.
- Internal data warehouse and cuboid reference: `docs/reference/data-warehouse.md`.
- Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining: Concepts and Techniques*. Chapter 3 discusses data warehouses, OLAP, data cubes, cuboids, and partial materialization. <https://datamineaz.org/textbooks/hanDataMiningConceptual.pdf>.